CS6711 SECURITY LABORATORY

EX. NO: 1

IMPLEMENTATION OF HASH FUNCTION – MD5

AIM:

To write a C program to implement the MD5 hashing technique.

DESCRIPTION:

MD5 processes a variable-length message into a fixed-length output of 128 bits. The input message is broken up into 512-bit blocks and padded so that its length is a multiple of 512 bits — a single '1' bit is appended, followed by zero bits, followed by a 64-bit representation of the original message length. MD5 operates on a 128-bit state made of four 32-bit words A, B, C and D, initialized to fixed constants. Each 512-bit block passes through 4 rounds of 16 operations each (using the non-linear functions F, G, H and I along with additive constants derived from the sine function and per-round left rotations) to update the state, and the final state (A||B||C||D) is the 128-bit MD5 digest.

ALGORITHM:

STEP-1: Read the input message and note its length in bits.

STEP-2: Pad the message (append a 1-bit, then 0-bits, then the 64-bit original length) so the total length is a multiple of 512 bits.

STEP-3: Initialize the four 32-bit state words A, B, C, D to the standard MD5 constants.

STEP-4: For every 512-bit block, run the 4 rounds of 16 non-linear, rotate-and-add operations (functions F, G, H, I) to update A, B, C, D.

STEP-5: Add the resulting A, B, C, D back into the running state after processing each block.

STEP-6: Concatenate the final A, B, C, D (in little-endian byte order) to form the 128-bit MD5 digest and print it in hexadecimal.

PROGRAM: (C)

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <math.h>

typedef union uwb
{
    unsigned w;
    unsigned char b[4];
} MD5union;

typedef unsigned DigestArray[4];

unsigned func0( unsigned abcd[] ){
    return ( abcd[1] & abcd[2]) | (~abcd[1] & abcd[3]);}
unsigned func1( unsigned abcd[] ){
    return ( abcd[3] & abcd[1]) | (~abcd[3] & abcd[2]);}
unsigned func2( unsigned abcd[] ){
    return abcd[1] ^ abcd[2] ^ abcd[3];}
unsigned func3( unsigned abcd[] ){
    return abcd[2] ^ (abcd[1] |~ abcd[3]);}

typedef unsigned (*DgstFctn)(unsigned a[]);

unsigned *calctable( unsigned *k)
{
    double s, pwr;
    int i;
    pwr = pow( 2, 32);
    for (i=0; i<64; i++)
    {
```

```
        s = fabs(sin(1+i));
        k[i] = (unsigned)( s * pwr );
    }
    return k;
}

unsigned rol( unsigned r, short N )
{
    unsigned mask1 = (1<<N) -1;
    return ((r>>(32-N)) & mask1) | ((r<<N) & ~mask1);
}

unsigned *md5( const char *msg, int mlen)
{
    static DigestArray h0 = { 0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476 };
    static DgstFctn ff[] = { &func0, &func1, &func2, &func3};
    static short M[] = { 1, 5, 3, 7 };
    static short O[] = { 0, 1, 5, 0 };
    static short rot0[] = { 7,12,17,22};
    static short rot1[] = { 5, 9,14,20};
    static short rot2[] = { 4,11,16,23};
    static short rot3[] = { 6,10,15,21};
    static short *rots[] = {rot0, rot1, rot2, rot3 };
    static unsigned kspace[64];
    static unsigned *k;
    static DigestArray h;
    DigestArray abcd;
    DgstFctn fctn;
    short m, o, g;
    unsigned f;
    short *rotn;
    union
    {
        unsigned w[16];
        char b[64];
    } mm;
    int os = 0;
    int grp, grps, q, p;
    unsigned char *msg2;

    if (k==NULL) k= calctable(kspace);
    for (q=0; q<4; q++) h[q] = h0[q];

    {
        grps = 1 + (mlen+8)/64;
        msg2 = malloc( 64*grps);
        memcpy( msg2, msg, mlen);
        msg2[mlen] = (unsigned char)0x80;
        q = mlen + 1;
        while (q < 64*grps){ msg2[q] = 0; q++ ; }
        {
            MD5union u;
            u.w = 8*mlen;
            q -= 8;
            memcpy(msg2+q, &u.w, 4 );
        }
    }

    for (grp=0; grp<grps; grp++)
    {
        memcpy( mm.b, msg2+os, 64);
        for(q=0;q<4;q++) abcd[q] = h[q];
        for (p = 0; p<4; p++)
        {
            fctn = ff[p];
            rotn = rots[p];
            m = M[p]; o= O[p];
            for (q=0; q<16; q++)
            {
                g = (m*q + o) % 16;
                f = abcd[1] + rol( abcd[0]+ fctn(abcd)+k[q+16*p]+ mm.w[g], rotn[q%4]);
```

```
                abcd[0] = abcd[3];
                abcd[3] = abcd[2];
                abcd[2] = abcd[1];
                abcd[1] = f;
            }
        }
        for (p=0; p<4; p++)
            h[p] += abcd[p];
        os += 64;
    }
    free(msg2);
    return h;
}

int main()
{
    int j,k;
    const char *msg = "The quick brown fox jumps over the lazy dog";
    unsigned *d = md5(msg, strlen(msg));
    MD5union u;

    printf("\t MD5 ENCRYPTION ALGORITHM IN C \n\n");
    printf("Input String to be Encrypted using MD5 : \n\t%s",msg);
    printf("\n\nThe MD5 code for input string is: \n");
    printf("\t= 0x");
    for (j=0;j<4; j++){
        u.w = d[j];
        for (k=0;k<4;k++) printf("%02x",u.b[k]);
    }
    printf("\n");
    printf("\n\t MD5 Encryption Successfully Completed!!!\n\n");
    return 0;
}
```

SAMPLE OUTPUT:

```
        MD5 ENCRYPTION ALGORITHM IN C

  Input String to be Encrypted using MD5 :
       The quick brown fox jumps over the lazy dog

  The MD5 code for input string is:
       = 0x9e107d9d372bb6826bd81d3542a419d6

        MD5 Encryption Successfully Completed!!!



  === Code Execution Successful ===
```

RESULT:

Thus the MD5 hashing algorithm was implemented successfully using C and the digest was verified to be correct.

EX. NO: 2

IMPLEMENTATION OF HASH FUNCTION – SHA-512

AIM:

To implement the SHA-512 hashing technique using a Java program.

DESCRIPTION:

SHA-512 is a member of the SHA-2 family of cryptographic hash functions published by NIST. It processes the input message in 1024-bit blocks and produces a fixed-length 512-bit (64-byte) message digest. Internally it maintains an 8-word (64-bit words) state and runs 80 rounds per block, combining bitwise logical functions, modular addition and fixed right-rotations/shifts, which makes it substantially more collision-resistant than the older MD5 and SHA-1 (which are both considered cryptographically broken today). Java's standard java.security.MessageDigest class provides a ready SHA-512 implementation via the algorithm name "SHA-512".

ALGORITHM:

STEP-1: Read the input message from the user.

STEP-2: Obtain a MessageDigest instance for the algorithm "SHA-512".

STEP-3: Pad the message internally as per the SHA-512 specification (this is handled automatically by the library).

STEP-4: Process the message in 1024-bit blocks, updating the 8-word internal state over 80 rounds per block using the SHA-2 compression function.

STEP-5: Call digest() to obtain the final 512-bit (64-byte) hash value and print it in hexadecimal.

PROGRAM: (Java)

```
import java.security.*;
import java.util.Scanner;

public class SHA512
{
    public static void main(String[] a) throws Exception
    {
        MessageDigest md = MessageDigest.getInstance("SHA-512");
        System.out.println("Message digest object info: ");
        System.out.println(" Algorithm = " + md.getAlgorithm());
        System.out.println(" Provider  = " + md.getProvider());

        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter the input string: ");
        String input = sc.nextLine();

        md.update(input.getBytes());
        byte[] output = md.digest();

        System.out.println("SHA512(\"" + input + "\") = " + bytesToHex(output));
    }

    public static String bytesToHex(byte[] b)
    {
        char hexDigit[] = {'0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F'};
        StringBuffer buf = new StringBuffer();
        for (int j=0; j<b.length; j++) {
            buf.append(hexDigit[(b[j] >> 4) & 0x0f]);
            buf.append(hexDigit[b[j] & 0x0f]);
        }
        return buf.toString();
    }
}
```

SAMPLE OUTPUT:

```
Message digest object info:
 Algorithm = SHA-512
 Provider  = SUN version 21

Enter the input string: run
SHA512("run") =
755986F37193A6D8C1EF1F254BA68213F3477A48DC1ACD78B551F82A54CF0E617B4AEE2E85C575D6DC0159396B9EE95BF
1FF52258DB13734959D92BB79

=== Code Execution Successful ===
```

RESULT:

Thus the SHA-512 hashing technique was implemented successfully using Java and the digest was verified against the standard test vector.

EX. NO: 3

IMPLEMENTATION OF HASH FUNCTION – SHA-1

AIM:

To implement the SHA-1 hashing technique using a Java program.

DESCRIPTION:

SHA-1 (Secure Hash Algorithm 1) is a cryptographic hash function that takes a message of length less than 2^64 bits and produces a 160-bit (20-byte) message digest. It is designed so that it is computationally infeasible to find two different input messages that hash to the same digest. A hash function like SHA-1 is used to compute a fixed-size alphanumeric string (a digest / fingerprint) that represents a file or piece of data, and it is meant to be unique and non-reversible. Note that SHA-1 is now considered cryptographically weak (practical collisions have been demonstrated) and is retained here only for lab/educational purposes — SHA-256/SHA-512 should be used in real applications.

ALGORITHM:

STEP-1: Read the input message from the user.

STEP-2: Obtain a MessageDigest instance for the algorithm "SHA1".

STEP-3: Pad the message internally as per the SHA-1 specification (handled automatically by the library).

STEP-4: Divide the padded message into 512-bit blocks; each block updates a 160-bit internal state (five 32-bit words A, B, C, D, E) over 80 rounds using SHA-1's logical functions and rotations.

STEP-5: Call digest() to obtain the final 160-bit hash value and print it in hexadecimal.

PROGRAM: (Java)

```
import java.security.*;
import java.util.Scanner;

public class SHA1
{
    public static void main(String[] a) throws Exception
    {
        MessageDigest md = MessageDigest.getInstance("SHA1");
        System.out.println("Message digest object info: ");
        System.out.println(" Algorithm = " + md.getAlgorithm());
        System.out.println(" Provider  = " + md.getProvider());

        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter the input string: ");
        String input = sc.nextLine();

        md.update(input.getBytes());
        byte[] output = md.digest();

        System.out.println("SHA1(\"" + input + "\") = " + bytesToHex(output));
    }

    public static String bytesToHex(byte[] b)
    {
        char hexDigit[] = {'0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F'};
        StringBuffer buf = new StringBuffer();
        for (int j=0; j<b.length; j++) {
            buf.append(hexDigit[(b[j] >> 4) & 0x0f]);
            buf.append(hexDigit[b[j] & 0x0f]);
        }
        return buf.toString();
    }
}
```

SAMPLE OUTPUT:

```
Message digest object info:
 Algorithm = SHA1
 Provider  = SUN version 21

Enter the input string: run
SHA1("run") = DF6AD19037C97987C4FF9792810C0E145356717C

=== Code Execution Successful ===
```

RESULT:

Thus the SHA-1 hashing technique was implemented successfully using Java and the digest was verified against the standard test vector.

EX. NO: 4

IMPLEMENTATION OF DIGITAL SIGNATURE STANDARD (DSA)

AIM:

To write a Java program to implement the Digital Signature Standard (DSA) algorithm.

DESCRIPTION:

The Digital Signature Algorithm (DSA) lets a signer prove authorship of a message without revealing their private key, and lets anyone verify that proof using only the signer's public key. DSA relies on a set of shared public parameters (a prime p, a prime divisor q of p-1, and a generator g of the order-q subgroup), a private key x (random, less than q) and the corresponding public key y = g^x mod p. To sign a message hash h, the signer picks a random per-message secret k, computes r = (g^k mod p) mod q and s = k^-1(h + x*r) mod q; the signature is the pair (r, s). A verifier recomputes v from the public key, g, r, s and h, and accepts the signature only if v equals r.

ALGORITHM:

STEP-1: Generate the global public parameters: a prime p, a prime divisor q of (p-1), and a generator g of the order-q subgroup mod p.

STEP-2: The signer picks a private key x (random, 0 < x < q) and computes the public key y = g^x mod p.

STEP-3: For a given message hash h, pick a random per-message secret k and compute r = (g^k mod p) mod q.

STEP-4: Compute the signature component s = k^-1 * (h + x*r) mod q. The signature is the pair (r, s).

STEP-5: To verify: compute w = s^-1 mod q, u1 = h*w mod q, u2 = r*w mod q, and v = ((g^u1 * y^u2) mod p) mod q.

STEP-6: If v equals r, the signature is valid; otherwise it is a forgery.

PROGRAM: (Java)

```
import java.util.*;
import java.math.BigInteger;

class DSAAlg {
    final static BigInteger one = new BigInteger("1");
    final static BigInteger zero = new BigInteger("0");

    public static BigInteger getNextPrime(String ans) {
        BigInteger test = new BigInteger(ans);
        while (!test.isProbablePrime(99))
        {
            test = test.add(one);
        }
        return test;
    }

    public static BigInteger findQ(BigInteger n)
    {
        BigInteger start = new BigInteger("2");
        while (!n.isProbablePrime(99))
        {
            while (!((n.mod(start)).equals(zero)))
            {
                start = start.add(one);
            }
            n = n.divide(start);
        }
        return n;
    }

    public static BigInteger getGen(BigInteger p, BigInteger q, Random r)
    {
        BigInteger h = new BigInteger(p.bitLength(), r);
```

```
        h = h.mod(p);
        return h.modPow((p.subtract(one)).divide(q), p);
    }

    public static void main (String[] args) throws Exception
    {
        Random randObj = new Random();
        BigInteger p = getNextPrime("10600");
        BigInteger q = findQ(p.subtract(one));
        BigInteger g = getGen(p,q,randObj);

        System.out.println("\nSimulation of Digital Signature Algorithm\n");
        System.out.println("Global public key components are:\n");
        System.out.println("p is: " + p);
        System.out.println("q is: " + q);
        System.out.println("g is: " + g);

        BigInteger x = new BigInteger(q.bitLength(), randObj);
        x = x.mod(q);
        BigInteger y = g.modPow(x,p);

        BigInteger k = new BigInteger(q.bitLength(), randObj);
        k = k.mod(q);
        BigInteger r = (g.modPow(k,p)).mod(q);

        BigInteger hashVal = new BigInteger(p.bitLength(), randObj);
        BigInteger kInv = k.modInverse(q);
        BigInteger s = kInv.multiply(hashVal.add(x.multiply(r)));
        s = s.mod(q);

        System.out.println("\nSecret information:\n");
        System.out.println("x (private) is: " + x);
        System.out.println("k (secret) is: " + k);
        System.out.println("y (public) is: " + y);
        System.out.println("h (message hash) is: " + hashVal);

        System.out.println("\nGenerating digital signature:\n");
        System.out.println("r is: " + r);
        System.out.println("s is: " + s);

        BigInteger w = s.modInverse(q);
        BigInteger u1 = (hashVal.multiply(w)).mod(q);
        BigInteger u2 = (r.multiply(w)).mod(q);
        BigInteger v = (g.modPow(u1,p)).multiply(y.modPow(u2,p));
        v = (v.mod(p)).mod(q);

        System.out.println("\nVerifying digital signature (checkpoints):\n");
        System.out.println("w is: " + w);
        System.out.println("u1 is: " + u1);
        System.out.println("u2 is: " + u2);
        System.out.println("v is: " + v);

        if (v.equals(r))
            System.out.println("\nSuccess: digital signature is verified! r = " + r);
        else
            System.out.println("\nError: incorrect digital signature");
    }
}
```

SAMPLE OUTPUT:

```
Simulation of Digital Signature Algorithm

Global public key components are:

p is: 10601
q is: 53
g is: 566

Secret information:

x (private) is: 49
k (secret) is: 9
y (public) is: 3848
h (message hash) is: 10644

Generating digital signature:

r is: 36
s is: 36

Verifying digital signature (checkpoints):

w is: 28
u1 is: 13
u2 is: 1
v is: 36

Success: digital signature is verified! r = 36

=== Code Execution Successful ===
```

RESULT:

Thus the Digital Signature Standard (DSA) algorithm was implemented successfully using Java and the generated signature was verified correctly.